Document Number	**SFS-CMPE-131 Sec xx**
Version	**1.0**
Status	**Draft**
Created By	**[Kyle Ringor and Evan Ha]**

OurSJSU

Software Functional Design Specification

Reviewers

Department	**Name/Title**
Engineering	Evan Ha
System Architecture	Christopher Nguyen
Product Management	Taha Khan
Marketing	Min Khant Htun
Quality Assurance (QA)	Rico Li
Engineering	Kyle Ringor

Team Member Contribution Record
Project Team: After Six
Deliverable: Software Functional Design Specification for OurSJSU
Each team member must participate in the development of this deliverable. As a team, review and agree on each member's contribution level and complete the table below before submission.

Team Member	Contribution Level	Brief Description of Contribution
Evan Ha	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 8 and 9
Christopher Nguyen	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 3, 10
Rico Li	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 11 and 12
Taha Khan	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 1 and 2
Kyle Ringor	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 4 and 5 and 7
Minh Khant Htun	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 6

Contribution Levels
Full Contribution (100%) – Fully participated and contributed to the deliverable; receives 100% of the team score.
Partial Contribution (50%) – Partially participated or contributed to the deliverable; receives 50% of the team score.
No Contribution (0%) – Did not participate or contribute to the deliverable; receives 0% of the team score.

Team Confirmation:
By submitting this document, the team confirms that the contribution levels reported above have been reviewed and agreed upon by the team members.

Table of Contents

1 Introduction
    1.1 Purpose of the Document
    1.2 Scope
    1.3 References
2 Design Goals
3 System Architecture
    3.1 High-Level Architecture
    3.2 Component Breakdown
4 Component Design
    4.1 Component Descriptions
    4.2 Component Interfaces
    4.3 Data Structures
5 Detailed Design
    5.1 Algorithms
    5.2 Class Design
6 User Interface Design
    6.1 Mockups
    6.2 UI Component Specifications
7 Database Design
        7.1 Database Schema
    7.2 Queries
8 Coding and Development Standards
9 Error Handling and Logging
10 Security Considerations
11 Test Design
    11.1 Unit Tests
12 Appendices

List of Tables

Table 3.1
Table 5.2

1 Introduction

1.1 Purpose of the Document

- Provide a detailed description of the software design, ensuring clarity for developers.
- Serve as a blueprint for implementing the system.
- The purpose of this document is meant to detail the Software Functional Design Specifications of OurSJSU. It will be a detailed blueprint for developers and QA testers.

1.2 Scope

- The functionality being described in this design specification is of OurSJSU.
- The functionality is an elaboration of the functional modules described in the System Functional Specification Document, from Module1:User Authentication to Module 9:MySJSU Redirection.

1.3 References

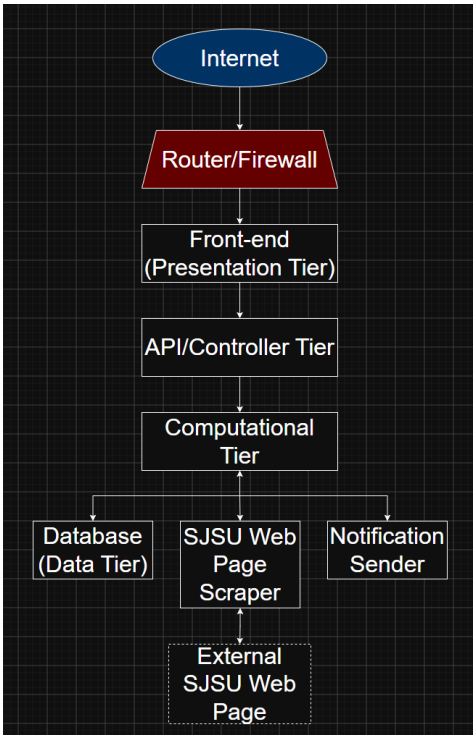
- Product Requirements Document (PRD)
- System Requirements Document (SAD)

2 Design Goals

- Usability: The Design creates an intuitive user experience due to the clean UI. Elements are added only when necessary and buttons are large enough to be visible.
- Performance: The design choices that highlight performance and efficiency are simple connections between the system and other APIs. When the user requests a feature that requires another API, very little is called except the API and a script that fetches the required information.
- Scalability: The system is scalable since many of the functions operate in the same way. The User clicks a button and a script brings information from an API. In this simple way, much functionality can be added later.
- Maintainability: The design facilitates future updates due to its scalability and simplicity. The functions can be updated easily due to their simple pseudocode layout and later features can be added quickly through copying existing functions and making simple modifications.
- Security: The system uses SSO (Single Sign On) to verify the User's details and fetch only the user's details. The system has role-based restrictions in order to prevent developers from viewing sensitive information.

3 System Architecture

3.1 High-Level Architecture



- Front-end (Presentation tier)
  - Student profile, degree roadmap, course search, watchlist
- API Gateway
  - Handles API requests and sends them to backend services
- Computational Tier
  - Authentication Manager: Manages login credentials
  - Search and Comparison Engine: Searches for and compares section times
  - Prerequisites Evaluator: Compares student's current and completed classes against the course catalog to determine whether the student is eligible
  - Roadmap: Takes in student information to generate a roadmap of future courses
  - Waitlist and availability monitor: Tracks waitlist statuses for tracked classes
  - Web scraper: Used to collect catalog updates (new and removed sections, changed section times, etc.) and detect seat availability changes

- Data Tier
  - Stores all necessary data
- External Systems
  - SJSU web pages are scraped for data
  - Notifications sent through Email, Discord, Slack, SMS, or other method of communication

3.2 Component Breakdown

- Authentication Module
  - Role: Manages user logins, validates active sessions, and processes logouts.
  - Interactions: Takes credentials or tokens and outputs session status to the frontend.
- Prerequisite Evaluator
  - Role: Determines whether student academic history meets course enrollment requirements.
  - Interactions: Receives student records and course requirements, then returns an eligibility result.
- Degree Planner
  - Role: Calculates percentage of degree completion and recommends eligible upcoming courses.
  - Interactions: Cross-references academic history with degree requirements and searches for prerequisite rules.
- Search and Comparison
  - Role: Filters course offerings and compares class schedules, instructors, and open seats.
  - Interactions: Accepts user search filters, queries the database, and returns matching sections.
- Watchlist Manager
  - Role: Saves targeted class sections and checks requirement eligibility upon addition to tracking list
  - Interactions: Connects student profiles to saved sections and interacts with prerequisite logic.
- Notification Sender
  - Role: Generates and sends alerts when saved section availability changes.
  - Interactions: Reads alert preferences and pushes messages to external delivery channels.
- Data Collector
  - Role: Periodically fetches updated course details and real-time seat availability.
  - Interactions: Collects external schedule data and writes updates to the database.

4 Component Design Kyle

4.1 Component Descriptions

- **Authentication**
  - Responsibilities: Handle demo login, create sessions, check expiration, and log out users.
  - Inputs: Login information or session token.
  - Outputs: Session token or student profile
  - Dependencies: Users, sessions, and student profiles.
- **Prerequisite Evaluation**
  - Responsibilities: Determine whether a student meets course requirements.
  - Inputs: Course requirements and student context.
  - Outputs: Eligible or ineligible
  - Dependencies: Academic history and requirement rules.
- **Degree Roadmap**
  - Responsibilities: Display course progress and recommend future courses.
  - Inputs: Student ID and major code.
  - Outputs: Roadmap, completion percentage, and recommendations.
  - Dependencies: Courses, major roadmaps, and prerequisite evaluation.
- **Search and Comparison**
  - Responsibilities: Find courses and compare sections by schedule, instructor, and availability.
  - Inputs: Search filters or selected section IDs.
  - Outputs: Matching courses and comparison results.
  - Dependencies: Courses, sections, meetings, and instructors.
- **Watchlist**
  - Responsibilities: Save sections and check prerequisites
  - Inputs: Student ID and section ID.
  - Outputs: Watchlist entry, warnings, or an error.
  - Dependencies: Student info, prerequisite evaluation, and database.
- **Notifications**
  - Responsibilities: Generate alerts for students watching a section.
  - Inputs: Availability update and notification preferences.
  - Outputs: Notification records and history.
  - Dependencies: Watchlists and notification preferences.

4.2 Component Interfaces

- **Internal Interfaces**
  - Frontend -> Backend: Search filters, student IDs, selected courses, and watchlist actions.
  - Backend -> Database: SQL queries, database rows, and saved changes.
  - Roadmap/Watchlist -> Prerequisite Evaluation: `CatalogCourse` and `StudentContext` objects.

    - Prerequisite Evaluation -> Calling Component:`EligibilityResult` containing status and explanations.
- **External Interfaces**.
    - `GET /api/courses/search`: Search using keyword, subject, instructor, and availability filters.
    - `POST /api/courses/compare`: Retrieve details for selected section IDs.
    - `POST /api/prerequisites/check`: Evaluate a course for a student.
    - `GET /api/roadmap/{major_code}`: Retrieve a student's roadmap and recommendations.
    - `GET/POST /api/watchlist`: Retrieve or add watched sections.
    - `DELETE /api/watchlist/{student_id}/{section_id}`: Remove the watched section.
    - `GET/PUT /api/notifications/preferences`: Retrieve or save notification settings.

4.3 Data Structures

- `CourseKey`
    - Main fields and types: `subject: str`, `catalog_number: str`
    - Purpose: Identify a course and produce an ID such as CMPE-130.
- `Meeting`
    - Main fields and types: `days: list[str]`, `start_time`, `end_time`, `location`, `dates`, `is_tba: bool`
    - Purpose: Describe one meeting.
- `Instructor`
    - Main fields and types: `name: str`
    - Purpose: Store an instructor's name.
- `Section`
    - Main fields and types: `term`, `class_number`, `section_number`, `course: CourseKey`, `meetings`, `instructors`, `open_seats: int | None`
    - Purpose: Represent a course section.
- `CatalogCourse`
    - Main fields and types: `course_key: CourseKey`, `title: str`, `description: str`, `units: int`, `requirement_text: str`, `prerequisite_rules`, `corequisite_rules`
    - Purpose: Represent catalog information.
- `RequirementRule`
    - Main fields and types: `kind: str`, `text: str`, `children: list`, `course_key: CourseKey`, `minimum_grade: str`, `concurrent: bool`, `operator: str`, `value`
    - Purpose: Represent requirements needed for enrolling into a class.
- `CourseAttempt`
    - Main fields and types: `course_key: CourseKey`, `status: str`, `grade: str | None`
    - Purpose: Record a completed or in-progress course.
- `StudentContext`
    - Main fields and types: `course_attempts: list[CourseAttempt]`, `earned_units: int`, `gpa: float`, `standing: str`, `major: str`, `placement_results: dict`, `permissions: list`
    - Purpose: Supply information for eligibility evaluation.
- `EligibilityResult`

- Main fields and types: `status: str`, `course: CourseKey`, `outcomes: dict[str, list[RuleOutcome]]`
- Purpose: Stores the eligibility decision.

5 Detailed Design Kyle

5.1 Algorithms

Algorithm 1: Prerequisite Evaluation

- Inputs: CatalogCourse and StudentContext.
- Output: EligibilityResult.

```
evaluate_rule(rule, student):
    IF rule needs all children to pass:
        evaluate children each child
        RETURN ineligible if any child fails
        RETURN unknown if any child is unknown
        RETURN eligible

    IF rule needs any child to pass:
        evaluate children each child
        RETURN eligible if any child passes
        RETURN unknown if any child is unknown
        RETURN ineligible

    compare the individual rule with student information
    RETURN eligible ineligible
```

Algorithm 2: Roadmap Recommendations

- Inputs: Student ID and major code.
- Outputs: Roadmap nodes, progress percentage, and recommended courses.

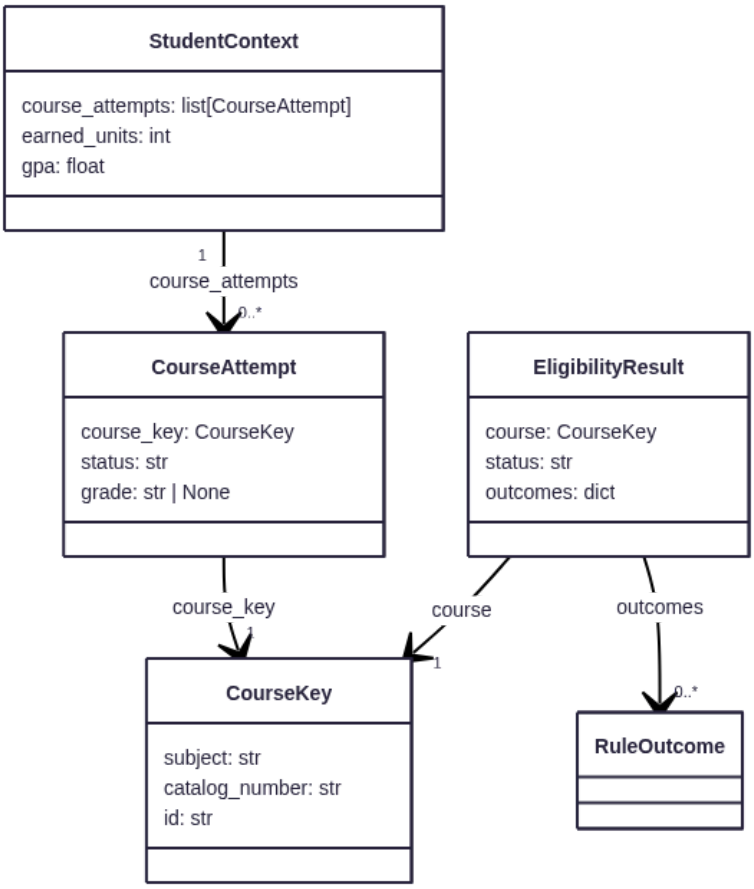
```
build_roadmap(student, major):
    Load student history and major requirements.
    Find the completed and in-progress courses.

    For each roadmap course:
        mark completed or in_progress
        otherwise evaluate eligibility

        IF eligible:
            add the course to recommendations
            count its direct dependent courses

    sort recommendations by dependent count, highest first
    calculate completed courses / roadmap courses × 100
    RETURN roadmap and recommendations
```

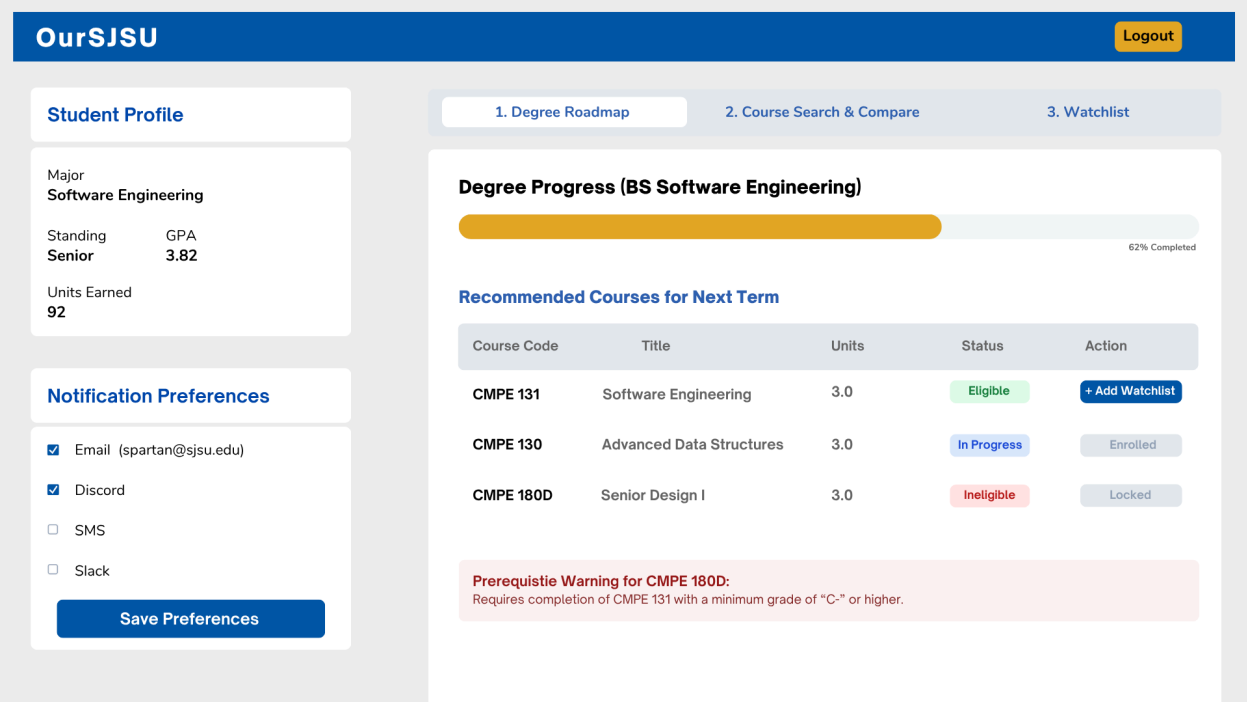
5.2 Class Design



- `AppState`
  - Main attributes: Student profile, active tab, search results, roadmap nodes, selected sections, watchlist, notification logs
  - Methods and responsibilities: `on_load()` will load page data; `run_search()` will refresh results; `load_roadmap()` will retrieve progress; watchlist methods will add/remove sections; `trigger_seat_drop()` will drop a seat.
- `OurSJSURequestHandler`
  - Main attributes: Database path and HTTP request information
  - Methods and responsibilities: `do_GET()`, `do_POST()`, `do_PUT()`, and `do_DELETE()` will handle requests; `_send_json()` will return results.
- `CourseKey`
  - Main attributes: Subject and catalog number
  - Methods and responsibilities: The `id` property will produce the course identifier.
- `Section`
  - Main attributes: Term, class number, course, meetings, instructors, seats
  - Methods and responsibilities: The `id` property will combine term and class number.

6 User Interface Design MKH

6.1 Mockups



6.2 UI Component Specifications

OurSJSU header → Navigation bar
Logout, "Add to watchlist" Button → Action Button
Notification Preferences → check boxes to preferred method of alerts
Degree Progress Bar → Visual Indicator for the student's progress
Prerequisites Warning Banner → Alert Panel

Navigation Tabs

- Tab 1: Degree Roadmap & Course Recommendations

  Goal: Provide students with a clear view of their degree progress and generate a prioritized list of eligible courses for upcoming terms.

- Tab 2: Course Search & Compare

  Goal: Enable students to search the active course catalog using filters, inspect section schedules, compare multiple sections side by side and view eligibility or add sections to their watchlist.

- Tab 3: Watchlist & Real-Time Seat Alerts

  Goal: Allow students to save specific course sections, track seat availability and provide real time alert notifications.

7 Database Design Kyle

7.1 Database Schema

The database already contains validated courses, sections, meetings, instructors, major roadmaps, and structured prerequisite/corequisite rules.

User and Academic Records

- `users`
  - Columns: `id` TEXT PK, `username` TEXT UNIQUE, `email` TEXT UNIQUE, `full_name` TEXT, `role` TEXT, `password_hash` TEXT, `created_at` TEXT
  - Relationships: One user will have multiple sessions and at most one student profile.
- `sessions`
  - Columns: `token` TEXT PK, `user_id` TEXT FK, `role` TEXT, `expires_at` TEXT
  - Relationships: Each session will belong to a user.
- `student_profiles`
  - Columns: `student_id` TEXT PK, `user_id` TEXT UNIQUE FK, `full_name` TEXT, `email` TEXT, `role` TEXT, `major` TEXT, `standing` TEXT, `gpa` REAL, `earned_units` INTEGER
  - Relationships: Each profile will connect to course attempts, watchlists, preferences, and logs.
- `student_course_attempts`
  - Columns: `id` INTEGER PK, `student_id` TEXT FK, `subject` TEXT, `catalog_number` TEXT, `status` TEXT, `grade` TEXT?
  - Relationships: Each attempt will belong to a student.

Courses and Sections

- `courses`
  - Columns: `id` TEXT PK, `subject` TEXT, `catalog_number` TEXT, `title` TEXT, `description` TEXT?, `units_min` REAL?, `units_max` REAL?, `prerequisite_text` TEXT?, `corequisite_text` TEXT?, `restrictions_text` TEXT?, `prerequisite_rule_json` TEXT?
  - Relationships: One course will have multiple sections and may appear in multiple roadmaps.
- `sections`
  - Columns: `id` TEXT PK, `term` TEXT, `class_number` TEXT, `section_number` TEXT, `course_id` TEXT FK, `title` TEXT, `mode` TEXT, `units` REAL?, `open_seats` INTEGER?, `notes` TEXT?
  - Relationships: Each section will belong to a course.
- `meetings`
  - Columns: `id` INTEGER PK, `section_id` TEXT FK, `days_json` TEXT, `start_time` TEXT?, `end_time` TEXT?, `location` TEXT?, `is_tba` INTEGER
  - Relationships: A section will have one or more meeting records when available.
- `instructors`

    - Columns: `id` INTEGER PK, `section_id` TEXT FK, `name` TEXT
    - Relationships: A section will have one or more instructor records when available.

Roadmaps, Watchlists, and Notifications

- `major_roadmaps`
    - Columns: `major_code` TEXT PK, `title` TEXT, `degree_type` TEXT, `description` TEXT?
    - Relationships: One roadmap will contain multiple course entries.
- `roadmap_courses`
    - Columns: `id` INTEGER PK, `major_code` TEXT FK, `course_id` TEXT FK, `term_order` INTEGER, `is_required` INTEGER
    - Relationships: Connects courses to major roadmaps.
- `watchlists`
    - Columns: `id` INTEGER PK, `student_id` TEXT FK, `section_id` TEXT FK, `term` TEXT, `added_at` TEXT, `active` INTEGER
    - Relationships: Connects students to watched sections.
- `notification_preferences`
    - Columns: `student_id` TEXT PK/FK, `email` TEXT?, `phone` TEXT?, `discord_webhook` TEXT?, `slack_webhook` TEXT?, `enabled_channels` TEXT
    - Relationships: Stores one preferences record per student when configured.
- `notification_logs`
    - Columns: `id` INTEGER PK, `student_id` TEXT FK, `channel` TEXT, `destination` TEXT, `event_type` TEXT, `message` TEXT, `status` TEXT, `sent_at` TEXT
    - Relationships: Stores multiple notification records for a student.

7.2 Queries

`?` depends on what the program is attempting to complete

Search for courses and their sections:

```
SELECT c.id, c.title, s.section_number, s.open_seats
FROM courses c
JOIN sections s ON s.course_id = c.id
WHERE c.subject = ?
  AND (c.id LIKE ? OR c.title LIKE ?)
ORDER BY c.id, s.section_number;
```

Retrieve student course history:

```
SELECT subject, catalog_number, status, grade
FROM student_course_attempts
```

```
WHERE student_id = ?
ORDER BY id;
```

Add or reactivate a watched section:

```
INSERT INTO watchlists
    (student_id, section_id, term, added_at, active)
VALUES (?, ?, ?, ?, 1)
ON CONFLICT(student_id, section_id)
DO UPDATE SET
    active = 1,
    added_at = excluded.added_at;
```

8 Coding and Development Standards Evan

- Code will use clear and descriptive names. Classes will use PascalCase, while variables, functions, and methods will use snake_case.
- Code will use consistent indentation, spacing, comments, and documentation.
- Python, SQLite, and a compatible IDE such as Visual Studio Code will be used for development.
- Git and Github will be used for version control. Developers will use separate branches, descriptive commit messages, and code reviews before merging changes into the main branch.

9 Error Handling and Logging Evan

- Error Handling:
  - The frontend will validate user input and display clear error messages.
  - The backend will validate requests and return appropriate HTTP error responses.
  - Database errors will be handled without losing or corrupting saved information.
  - If an external service or scraper fails, the system will log the error, retry when appropriate, and use cached data when available.
- Logging:
  - The system will use DEBUG, INFO, WARNING, and ERROR log levels.
  - Log entries will include a timestamp, log level, component, and message.
  - Authentication failures, database errors, scraper failures, and notification failures will be recorded.
  - Passwords, session tokens, and private student information will not be written to logs.

10 Security Considerations Christopher

- **Authentication & Session Management**
  - Hash and store user passwords safely.
  - Generate secure session tokens with expiration tracking.
- **Authorization & Access Control**
  - Verify user identity on every request to prevent unauthorized access to student records.
  - Enforce role-based limits to protect restricted actions.
- **Data Integrity & Injection Prevention**
  - Use parameterized database queries to prevent SQL injection.
  - Validate all incoming request data before execution.
- **Data Privacy & Logging Security**
  - Encrypt data in transit using HTTPS/TLS.
  - Strip passwords, tokens, and private student data from system logs.
- **External Integration & Traffic Security**
  - Validate webhook destination URLs to prevent server-side request manipulation.
  - Apply rate limiting on API endpoints to prevent brute-force attacks.

11 Test Design

11.1 Unit Tests

Authentication and Session Management:

Given valid SJSU credentials, the authentication service returns a session token containing the user ID, role, and expiration time. Given invalid credentials, it returns an error, creates no session, and records the failed attempt without logging the password.

Prerequisite Evaluation:

For completed, in-progress, missing, and conflicting prerequisite rules, the evaluator returns eligible, ineligible, or unknown as appropriate. AND rules fail when any required child rule fails; OR rules pass when at least one child rule passes.

Degree Roadmap:

Given a student profile and major code, the roadmap service marks completed and in-progress courses correctly, calculates progress as completed roadmap courses divided by total roadmap courses, and recommends eligible courses.

Course Search and Comparison:

Given keyword, subject, instructor, and availability filters, the search service returns only matching sections. Comparison results contain the course, instructor, meeting time, location, units, and seat availability for every selected section.

Watchlist:

Adding a section creates or reactivates one watchlist entry. Removing a section deactivates the entry. Duplicate additions do not create duplicate records, and prerequisite warnings are returned when applicable.

Availability Monitoring:

When a section changes from closed to open, the monitor creates one availability-change event. Unchanged sections do not create duplicate events, and scraper failures preserve the last valid cached data.

Notification Dispatch:

For an availability-change event, the service selects the student's enabled channels and creates a delivery record for each channel. A provider failure is logged and retried without stopping delivery to other channels.

Data Model and API Validation:

Required fields, foreign-key relationships, unauthorized student access, invalid section IDs, and malformed request bodies are rejected with clear errors and without corrupting saved data.

12 Appendices

- Glossary of terms used in the specification.

Availability Change

An event created when the recorded seat status of a watched section changes.

Catalog Course

A course definition containing its title, units, description, prerequisites, and corequisites.

Course Attempt

A student's record of a completed or in-progress course and, when available, the grade.

Eligibility Result

The prerequisite-evaluation response indicating whether a student is eligible, ineligible, or unknown for a course.

Major Roadmap

The ordered set of courses and requirements used to show degree progress and recommend future courses.

Notification Preference

A student's selected delivery channels and destinations for alerts.

Student Context

The academic information used during eligibility evaluation, including course history, units, GPA, standing, and major.

Watchlist

A student's saved list of course sections that OurSJSU monitors for changes.

Web Scraper

A scheduled service that collects course and seat-availability data from SJSU web pages.

- Additional Diagrams: Any supplementary diagrams that support the design

High-level MVC/client-server architecture showing the Web Interface, Backend/API, Database, SJSU data source, and notification providers.

Course availability sequence showing the scraper, availability monitor, database, watchlist, and notification dispatch service.

Prerequisite-evaluation flow showing StudentContext and CatalogCourse entering the evaluator and EligibilityResult returning to the roadmap or course-search component.

Degree-roadmap flow showing completed courses, prerequisite checks, progress calculation, and course recommendations.

- Review meeting minutes (time, attendees, reviewer/review comments, action items/PICs).

Meeting purpose:

Review the Software Functional Design Specification against the Product Requirements Document and System Architecture Document.

Time: 2026/09/15 1:30-2:45

Attendees: Min Khant Htun, Taha Khan, Rico Li, Christopher Nguyen, Kyle Ringor, Evan Ha

Reviewer comments:

Confirm that each high-priority PRD requirement is represented by a functional component, interface, data structure, or test case.

Confirm that authentication, student-data privacy, cached data behavior, and notification failures are addressed.

Action items and person in charge:

1. Verify the final UI mockups and add their location to Section 7.Confirm the SJSU data source and notification provider assumptions.

PIC: Kyle

2. Run the unit and integration tests listed in Section 11 and record the results before submission.

PIC: Rico